

The Personal Event Bus

User-owned middleware for ambient digital-life events: an append-only, schema-normalized local event log with typed, permissioned subscriptions for consumer applications.

p-rick research program

p-rick — an independent research program on user-owned computing and the economics of inattention.

The Personal Event Bus

SUBThe Personal Event Bus

p-rick research program

Working Paper P-001 · v1.0 · October 7, 2026 · CC BY 4.0

Abstract — A person’s digital life emits a continuous stream of semantically rich events — payment notifications, e-receipts, delivery updates, appointment confirmations, subscription renewals — yet this stream is discarded or trapped per-application. Operating systems expose the raw bytes (a notification shade), cloud automators route individual triggers through vendor servers, personal-data vaults store documents at rest, and vertical apps each rebuild one-off capture for a single purpose. No shipping product, standard, or research system combines all five elements this layer requires — ambient capture, durable append-only storage, schema normalization, typed third-party subscriptions, and user-held ownership. Adjacent systems each satisfy a strict subset: Solid pods store without capturing; IFTTT/Zapier route without persisting or owning; Home Assistant generalizes event-bus architecture brilliantly but for the smart home, not personal records; Microsoft’s MyLifeBits captured everything durably in research but without schemas, subscriptions, or a permission model. We specify the missing layer — the *personal event bus* — as a formal object: an append-only local event log with a normalized event schema, pluggable capture adapters, and a permission calculus governing third-party subscriptions. We position the bus against fifteen adjacent systems and show each occupies a different quadrant of the design space, leaving the center unoccupied. We analyze why the gap persists (infrastructure-first sequencing failures, a 2019 platform-policy chilling event, absorption of the semantics into closed OS features, and an unnamed category), demonstrate that feasibility is proven piecewise by shipping verticals (PocketVeto as an existence proof for on-device notification parsing), and propose vertical-first adoption mechanics. We close with the regulatory argument: current portability law (GDPR Article 20, the DMA) is blind to *event streams*, and the personal event bus is the natural receiving endpoint for an event-level right to portability. **Keywords:** personal event bus, local-first, ambient computing, data portability, middleware, privacy —

Keywords: personal event bus, local-first, ambient computing, data portability, middleware, privacy

A person’s digital life emits a continuous stream of semantically rich events — payment notifications, e-receipts, delivery updates, appointment confirmations, subscription renewals — yet this stream is discarded or trapped per-application. Operating systems expose the raw bytes (a notification shade), cloud automators route individual triggers through vendor servers, personal-data vaults store documents at rest, and vertical apps each rebuild one-off capture for a single purpose. No shipping product, standard, or research system combines all five elements this layer requires — ambient capture, durable append-only storage, schema normalization, typed third-party subscriptions, and user-held ownership. Adjacent systems each satisfy a strict subset: Solid pods store without capturing; IFTTT/Zapier route without persisting or owning; Home Assistant generalizes event-bus architecture brilliantly but for the smart home, not personal records; Microsoft’s MyLifeBits captured everything durably in research but without schemas, subscriptions, or a permission model. We specify the missing layer — the *personal event bus* — as a formal object: an append-only local event log with a normalized event schema, pluggable capture adapters, and a permission calculus governing third-party subscriptions. We position the bus against fifteen adjacent systems and show each occupies a different quadrant of the design space, leaving the center unoccupied. We analyze why the gap persists (infrastructure-first sequencing failures, a 2019 platform-policy chilling event, absorption of the semantics into closed OS features, and an unnamed category), demonstrate that feasibility is proven piecewise by shipping verticals (PocketVeto as an existence proof for on-device notification parsing), and propose vertical-first adoption mechanics. We close with the regulatory argument: current portability

law (GDPR Article 20, the DMA) is blind to *event streams*, and the personal event bus is the natural receiving endpoint for an event-level right to portability.

1 Introduction

Consider an ordinary Tuesday. A bank pushes a notification: “Rs. 1,199 debited by UPI mandate to NATIONWIDE-INSURE”. A pharmacy app emails a receipt for a refill. A flight app notifies a gate change. A subscription renews silently on a card. Each of these is a *structured, consequential, machine-parseable event* about your life. What happens to them?

The bank’s notification scrolls away after a glance. The email sits in an inbox of ten thousand. The renewal charge surfaces a month later in a statement PDF. The events are *ambient* — they arrive unprompted, unauthenticated by you, and semantically dense — and the computing ecosystem’s answer to them is, functionally, *amnesia*. Weiser and Brown’s “calm technology” program anticipated ambient information that stays in the periphery until it matters; what it did not anticipate is that the infrastructure of *remembering* would never be built.

This is a strange state of affairs, because every ingredient needed to remember is present. Phones can observe their own notifications (Android has exposed `NotificationListenerService` since API 18). Inboxes are machine-readable via IMAP. On-device models can parse free text into structured records without a byte leaving the phone. Storage is effectively free. Yet no mainstream system composes these ingredients into a durable personal memory. The result is a systemic asymmetry: *platforms* keep exhaustive behavioral records of users, while *users* keep almost none of the events that constitute their financial and civic lives. Institutional memory without personal memory is the defining data asymmetry of the current era — and most responses to it (privacy law, delete buttons, vaults) treat data as a *stock* while ignoring that for the individual, the meaningful unit is a *flow*.

This paper names and specifies the missing layer. We call it the **personal event bus**: a user-owned, on-device middleware that (a) captures ambient events through pluggable adapters, (b) normalizes them into a versioned schema, (c) appends them to an encrypted, append-only local log, and (d) exposes typed, revocable subscriptions to local consumer applications — with all inference on-device.

The contributions of this paper are:

1. **A naming and bounding of a barely-named software category.** Exact-term searches for the concept return nothing but noise (Appendix A documents the search protocol); the category has no incumbents because it has no stable name. We give it one and delimit it from fifteen adjacent systems (Section 3).
2. **A formal specification** of the event model, log, and permission calculus, sufficient for independent interoperable implementations (Section 4).
3. **An architecture** for capture adapters, parser plugins, and consumer APIs, with a privacy threat model that treats *parser plugins* as the primary trust boundary (Sections 5–6).
4. **An explanation of persistence** for the gap: infrastructure-first sequencing failures across a decade of attempts (Solid, Databox, HAT, digi.me), a 2019 platform-policy event that chilled the SMS-parsing generation, and the chicken-and-egg problem of middleware without consumers (Section 7).
5. **Adoption mechanics** grounded in a shipping case study — PocketVeto, an Android application whose payment-notification capture is an existence proof for one bus adapter (Section 8).
6. **A regulatory extension:** personal data portability under GDPR Article 20 and the DMA is defined over data *provided by* the user, not event streams; we argue the bus is the correct

receiving endpoint for event-level portability and propose the standard’s role in that future (Section 9).

We are deliberate about what this paper is not. It is not a proposal for another cloud automation service, a personal knowledge base, or a research prototype awaiting a data-warehouse backend. The position is narrower and more infrastructural: the bus is to personal events what the print spooler is to documents — boring, local, universally assumed, and therefore never built on purpose.

2 The problem: ambient events, institutional amnesia

Define an **ambient personal event** as a machine-delivered record of a real-world transition involving the user — a payment, a shipment, an appointment, a renewal, a price change, a recall — that arrives *without user initiation* and carries *parseable structure* (sender identity, timestamp, amounts, counterparties, references).

Ambient events differ from the personal data that privacy scholarship usually concerns itself with in three ways:

- **Provenance.** They are *pushed to* the user by counterparties (banks, merchants, carriers, governments). The user is the recipient, not the requester. This matters legally: GDPR Article 20 portability covers data “provided by” the data subject, and an event pushed by a bank sits in a gray zone.
- **Temporality.** Their value decays. A payment notification is maximally actionable within seconds (fraud), useful within days (budgeting), and merely archival within months. A stock-oriented model (export, archive, vault) systematically arrives too late.
- **Composition.** No single source suffices. A complete picture of a household’s financial events is scattered across one bank’s SMS channel, three UPI apps’ notifications, two card issuers’ emails, and a merchant’s e-receipts. Any single-source tool is structurally partial.

The consequences of discarding this stream are not hypothetical. They are the subject of two companion papers in this program: subscription autopays persist because renewal events are never observed and compared (P-002: unclaimed consumer rights, of which silently renewed subscriptions are a leading instance, run to billions per year); and price histories vanish because purchase events are never accumulated, making personal inflation uncomputable (P-003). Both downstream failures trace to the same upstream absence: **there is no place for the events to land.**

3 The design space: fifteen adjacent systems, none in the center

We surveyed the landscape across products, open-source projects, and research systems (protocol in Appendix A). The result is summarized in Table 1, along five axes that define the design space: *capture* (does it ingest ambient events?), *durability* (does it persist them?), *semantics* (does it normalize structure?), *consumers* (can third-party apps subscribe?), and *ownership* (is the data user-held and local?).

Four observations follow.

First, the center is unoccupied. Every existing category is strong on one or two axes and absent on the rest. Solid stores documents but captures nothing ambient. IFTTT routes triggers but persists nothing and routes them through vendor cloud. Notification loggers capture everything but normalize nothing and share with no one. Vertical finance apps capture and normalize but only for their own schema and their own consumption.

Table 1: Table 1 — The design space.

System / category	Capture	Durability	Semantics	Consumers	Ownership
Solid pods / Inrupt	—	✓ (documents)	linked data	web apps (ACL)	user-held
IFTTT / Zapier	✓ (cloud triggers)	—	per-service	actions	vendor-held
Node-RED / n8n / Huginn	✓ (feeds, incl. IMAP)	✓	ad hoc	flows	self-hosted
Home Assistant	✓ (device/integration events)	✓ (recorder)	typed per integration	local automations	user-held
MyLifeBits (Bell & Gemmell, MSR 2001–07)	✓ (lifelogging)	✓	— (raw artifacts)	research UI	local
KDE Connect / Pushbullet	✓ (mirror)	—	—	other devices	paired
Apple Shortcuts / Automations	partial	—	shallow	Apple only	sandboxed
Apple Intelligence / Gemini	✓ (on-device)	—	vendor AI	first-party	vendor
Notification-history apps (Notlog, Catch Notify)	✓	✓ (raw text)	—	single app	local
Vertical capture apps (Walnut, ET Money, PocketVeto)	✓	✓	one vertical	own app only	varies
Tasker / AutoNotification	✓	— (variables)	ad hoc	own tasks	local
Databox / HAT / digi.me (research)	✓ (partner APIs)	✓	mixed	consented firms	box/vault
Local-first CRDT apps (Kleppmann et al.)	—	✓	documents	collaborators	local-first
Google Takeout	— (batch)	✓ (files)	raw	manual	export-only
Personal event bus (proposed)	✓ (ambient)	✓ (append-only)	✓ (normalized)	✓ (typed, local)	✓ (user)

Second, the two OS vendors are converging on a closed subset of the center. As of October 2026, Apple Intelligence summarizes notifications on-device and Android’s Gemini-era stack does similarly (vendor documentation; these are moving targets — the urgency argument below degrades if either ships durable, open notification subscriptions, and we would welcome the obsolescence). Both are currently ephemeral (no durable log), first-party-only (no third-party subscriptions), and non-interoperable (no schema export). The semantic layer over notifications is being built — *as a proprietary feature, not as infrastructure*. The window for an open version is closing.

Third, the closest relatives died of sequencing — or scoped themselves elsewhere. Imperial College’s Databox (Haddadi et al., 2016) proposed a personal, networked box for consented data processing, with trusted execution and per-app data access — and has been dormant since ~2018. The HAT and digi.me lineage similarly aimed at personal data marketplaces and pivoted or wound down (digi.me moved to medical records). ProjectVRM, Searls’ long-running customer-empowerment program, concluded as recently as 2025 that the infrastructure future it advocated “is still not here.” Home Assistant proves local, user-owned event-bus architecture is buildable and beloved — but for devices, never for personal records; MyLifeBits proved a decade of durable personal capture in research — without schemas, subscriptions, or permissions; the Context Toolkit (Dey & Abowd 2001) supplied typed context subscriptions for ubicomp research a quarter-century ago. These were not wrong ideas; they were *infrastructure-first* ideas, shipped before any consumer application created demand, or *adjacent-domain* ideas that never crossed into personal records. We return to this in Section 7.

Fourth, an exact-term search for the category returns noise. Searches for “personal event bus” and “ambient ledger” surface no product, standard, or project claiming the concept (Appendix A). The category is not contested; it is *unnamed*. Naming is cheap coordination — we do not claim that names cause categories, only that an unnamed category cannot be found by the founders and researchers who would build it.

4 Formal specification

We now specify the bus precisely enough for interoperable implementations. The specification is deliberately minimal; it standardizes the *envelope*, not the payload taxonomies, which are left to per-domain schema packs.

4.1 Event model

Let E be the set of ambient events. An event e in E is a tuple:

```
e = (id, source, type, occurred_at, captured_at, payload, provenance, confidence,
lineage)
```

- **id:** content-addressed identifier (e.g., BLAKE3 hash of the canonical serialization).
- **source:** capture adapter identity — {`adapter`, `channel`, `counterparty_hints`} (e.g., {`notif_listener`, `whatsapp://notification`, `sender=bank`}, {`imap`, `inbox/messages/4123`, `from=payments@stripe`}).
- **type:** URI-namespaced event type from a versioned schema pack (e.g., `finance/payment/v1`, `logistics/shipment/v1`, `calendar/appointment/v1`).
- **occurred_at / captured_at:** the real-world event time (parsed or inferred) and the local capture time. The difference is a first-class field; downstream applications must be able to discount late or backfilled captures.
- **payload:** typed record per the schema pack — for `finance/payment/v1`: {`amount`, `currency`, `direction`, `counterparty`, `mandate_hints[]`, `references[]`}.

- **provenance:** cryptographic pointer to the raw artifact (the raw notification bytes, the RFC-822 message) retained in a raw blob store, enabling re-parsing and audit.
- **confidence:** parser-emitted scalar plus per-field confidences. Ambiguity is a *first-class* state, never silently resolved.
- **lineage:** if this event was derived from another (dedupe, merge, enrichment), the parent ids.

4.2 The log

The bus is an **append-only log** $L = e_1, e_2, \dots$, physically stored in an encrypted local database (SQLite as reference), with three structural invariants:

1. **Immutability.** Events are never updated; corrections are new events with lineage. This is required because the log is evidence — P-002’s claim engine consumes it to assert legal entitlements, and evidence must be append-only to be auditable.
2. **Deterministic ordering and tamper evidence.** By `captured_at`, with `id` as tiebreaker. Each event record carries a running hash over the canonical serialization of its predecessor (a per-record chain), and the chain head is periodically checkpointed — anchored, optionally, to any durable external store the user chooses (a personal cloud drive, a paper copy). Verification replays the chain; neither construction replaces cryptography with trust in a remote party.
3. **Retention policy as data, with an evidence-grade mode.** Per-source retention rules (including infinite) are part of the bus’s user-facing configuration, not a storage implementation detail. Because P-002’s claim engine treats the log as legal evidence, sources may be marked *evidence-grade*: deletions in those classes are tombstoned rather than erased (content removed, chain continuity preserved), reconciling GDPR Article 17 erasure with auditability. We state the compromise rather than hide it: tombstoning shifts what can be *proven* from content to continuity, and downstream consumers (P-002 §5.5) may rely only on the guarantees stated here.

4.3 Subscriptions and the permission calculus

A **consumer** is a local application that reads the bus. Consumers do not get raw log access. They hold **subscriptions**:

```
sub = (consumer_id, type_filter, field_mask, since)
```

- **type_filter:** schema-pack types (a ledger app subscribes to `finance/payment/v1`).
- **field_mask:** the subset of payload fields the consumer may read — a calendar app gets `occurred_at` and `counterparty_hints`, not `amount`.

The permission model is **capability-based and revocable**: subscriptions are granted by an explicit user action (an OS-style permission sheet), can be narrowed or revoked without consumer consent, and are enforced by the broker process, not by consumer goodwill. Formally, the information available to a consumer is:

$$I(c, t) = \text{Prod_mask}(L \text{ filtered by } \text{type_filter}, \text{ up to } t)$$

where `Prod_mask` is the field-mask projection. This is the calculable privacy property we consider load-bearing: a consumer’s access is *bounded in field content* by its subscription, by construction. We state the residual channels honestly: two consumers with complementary masks can collude to reconstruct records, and event *arrival times* are a side channel that survives even a null

mask (the fact that *something* happened is itself information). The broker can mitigate (per-consumer delivery pacing; dummy traffic at the user’s discretion) but not eliminate these; the claim is bounded, not zero.

4.4 Parser plugins and schema packs

Parsing (raw notification bytes $\rightarrow$ typed event) is performed by **plugins** — sandboxed, capability-less code (WASM as the reference target) with no network access, selected by a routing table from (**channel**, **sender**) to plugin. Schema packs are versioned collections of types + plugins, maintained as community artifacts (the analogue of a blocklist/adblock subscription). This is the standard’s *open commons* layer: the bus is the substrate; the packs are the collaborative corpus.

5 Architecture

The reference runtime has five components:

1. **Capture adapters.** Android `NotificationListenerService` (proven in production by verticals); IMAP IDLE for e-receipts (universal, platform-neutral); SMS where carrier/platform policy permits (India’s UPI SMS trail being the canonical high-value case); manual share-sheet fallback for locked platforms.
2. **Raw blob store.** Verbatim artifacts (notification bytes, MIME messages), content-addressed, encrypted, governed by retention policy.
3. **Parser router + plugins.** The routing table and sandboxed plugin set; plugins emit typed events with confidences and never touch the network.
4. **The log and broker.** The append-only store and the process that enforces subscriptions, exposes a local typed API (gRPC-over-UDS or in-process), and emits change notifications to consumers.
5. **User surfaces.** A permission manager (grants, masks, retention), an event inspector (audit surface), and export (the log in an open serialization — the interoperability escape hatch).

A deliberate omission: there is **no cloud component, no account, and no telemetry**. Sync (multi-device) is out of scope for v1 and, when it arrives, must be end-to-end encrypted with the log’s hash chain intact across replicas.

6 Threat model and trust boundaries

The interesting adversary is not the network observer (the bus is local and encrypted at rest) but the **components**:

- **A hostile consumer app** wants events it is not subscribed to. Mitigated by construction: it can only call the broker, which enforces the mask. The consumer never handles raw artifacts.
- **A hostile parser plugin** wants to exfiltrate. It runs sandboxed with no network capability; its only output channel is typed events, which are themselves available for audit in the user-facing inspector. The residual risk — encoding data *into* typed-event fields (e.g., smuggling a phone number into **counterparty** so that a subscribed consumer reads it out) — cannot be bounded by the mask, which constrains *consumers*, not *writers*. It is mitigated by audit (the inspector shows any plugin’s complete output, unmasked) and by plugin provenance (signed packs with reputations). Parser-output audit is an open UX problem, not a solved one, and we flag it as such.

- **A compromised host OS** is out of scope; no local-first design survives it. We note, however, that the bus *reduces* the value of OS compromise for mass surveillance relative to cloud alternatives, because no central collection point exists.
- **Counterparty countermeasures** (banks obfuscating notification text to defeat parsing) are a live, ongoing adversarial co-evolution. The provenance store (keeping raw artifacts) is the hedge: parsers are replaceable retroactively, so counterparty obfuscation degrades gracefully rather than catastrophically.

The privacy calculus literature (Culnan & Armstrong 1999; Dienlin & Trepte 2015) predicts adoption of disclosure-gated systems from perceived benefit versus risk. The bus improves the calculus by construction: disclosure is field-scoped, local, and revocable — the three levers the calculus identifies. We conjecture (and a deployment study should test) that this makes ambient capture *more* adoptable than the status quo, where users grant blanket permissions to vertical apps with unbounded retention.

7 Why the gap persists

Four independent causes, each instructive:

1. **Infrastructure-first sequencing.** Solid, Databox, HAT, and digi.me all shipped the substrate and waited for applications. The applications never came (cold start), and the projects pivoted or died. The lesson is not “the layer is unwanted” but “the layer cannot lead.” Middleware gets adopted *retrofitted onto a killer consumer*, not procured in advance. (This is the same pattern that killed general-purpose rdf stores in the 2000s while RSS — a narrower event format — saturated the web.)
2. **The 2019 chilling event.** Google Play policy changes (October 2019) restricted SMS and call-log access to a narrow class of apps, effectively ending the SMS-parsing personal-finance generation (Walnut et al.) and teaching the ecosystem that ambient capture is a policy-hostile category. Notification-based capture survived as a quieter niche — which is where PocketVeto lives.
3. **Platform absorption in progress.** The OS vendors are building the semantic layer (Apple Intelligence, Gemini) as proprietary features. When the platform provides a *closed* version for free, an *open* version must justify itself on durability, interoperability, and audit — arguments that only land once the closed version’s limitations are felt.
4. **The unnamed-category problem.** Search for the concept and you find nothing, so founders and researchers cannot find prior art, competitors, or vocabulary. Categories are coordination devices; without the name, no one coordinates. This paper’s first contribution is fixing that.

8 Adoption mechanics: vertical-first, retrofitted substrate

The Databox lesson dictates the strategy: ****the bus ships as the *engine* of a consumer application that would exist anyway, then reveals itself as infrastructure.****

The case study is PocketVeto (this repository’s companion product): an Android autopay/subscription tracker whose capture path is exactly one bus adapter (payment notifications → typed payment events) with one consumer (the app’s ledger). In bus terms, PocketVeto is **finance/payment/v1** schema pack plus a radar UI. The retrofit path is literal: extract the adapter and broker from the app into a library, add the permission sheet and event inspector, and the app’s capture layer *becomes* the bus, gaining a second consumer the moment a second schema pack (e-receipts) is enabled.

The sequencing hypothesis is testable and falsifiable: **no general personal-data substrate will reach consumer adoption without first surviving as the hidden engine of a specific app that solves a specific money problem.** If true, it explains every failure in Section 3 and predicts the only viable route for the bus. If false, the bus remains a specification waiting for a vendor — and the closed OS absorption wins by default.

Distribution realities are acknowledged: sideload friction (Play Protect false positives on notification-listening wrappers — documented in this repo’s own release engineering), and the iOS constraint (Section 9), which confines the reference platform to Android + desktop e-mail.

9 Platform feasibility and the regulatory argument

Platforms. Android: full-fidelity capture is feasible today (`NotificationListenerService`; self-rebind after OS unbinds — a production lesson documented in `PocketVeto v1.5.3`). iOS: no third-party notification API; the bus degrades to share-sheet manual ingestion and IMAP e-receipt capture. Desktop: IMAP is the universal, durable channel, and e-receipts cover a large fraction of consumer-commerce events.

Regulation. GDPR Article 20 grants portability over data “provided by” the data subject — historically read as account data and uploads. A payment notification is pushed by a counterparty; whether it is “provided by” the user is unsettled. The DMA (Art. 6) imposes portability obligations on gatekeepers, but again over *held data*, not *event streams*. The gap is doctrinally interesting: the events that matter most for consumer empowerment (transactions, renewals, receipts) are precisely the ones the portability framework is least sure about. We propose the standard the bus implements as the *receiving endpoint* for an event-level right to portability: if regulators ever require counterparties (banks, merchants) to emit machine-readable event streams to a user-designated endpoint, the bus — schema-versioned, append-only, locally enforced — is, to our knowledge, the only proposed architecture that could receive them without becoming a cloud honeypot. The bus is thus not only a product thesis but a *regulatory completion*: the missing receiving side of portability.

10 Limitations and honest risks

We state the strongest objections rather than the kindest:

1. **iOS lockout.** The platform where affluent consumers are blocks ambient capture. The bus is Android+desktop-first; if Apple ships an open notification-subscription API (currently: no public plans), the calculus changes.
2. **OS-vendor absorption.** “The OS already does this” will be the reviewer’s one-line kill. The counter — closed, ephemeral, first-party-only — is real but must be *demonstrated*, not asserted, as absorption proceeds.
3. **Chicken-and-egg residual risk.** Even vertical-first, the bus competes with every vertical app’s *private* capture stack. The library must be strictly easier than the one-off.
4. **Counterparty obfuscation.** Banks and UPI apps have already degraded notification text; parsing is an arms race. The provenance hedge makes losses recoverable, not preventable.
5. **Scope discipline.** The temptation to add inference, dashboards, and AI features to the bus is the death of it. The bus’s product surface is the permission sheet, the inspector, and the log. Everything else is a consumer.

11 Roadmap and open problems

- **v0 (exists):** single-app embedded capture (PocketVeto’s adapter as the seed).
- **v1:** extracted library — broker, log, permission sheet, **finance/payment/v1** and **commerce/ereceipt/v1** packs, IMAP adapter, export.
- **v2:** third-party consumer API stability; hash-chained checkpoints; pack registry as a community artifact.
- **Open problems:** multi-device sync with chain preservation; formal verification of the broker’s mask enforcement; a usable language for users to *read* what a subscription means (permission UX is the real bottleneck); plugin update distribution without a central authority (web-of-trust signing of schema packs).

12 Conclusion

Personal computing keeps exhaustive institutional records and gives individuals amnesia. The ambient event stream that constitutes a person’s financial and civic life arrives, is glanced at, and evaporates — not because capture, storage, or parsing are hard, but because no layer exists whose job is to remember it *for the user, under the user’s terms, in a form other software can use*. We have specified that layer, shown that the design space around it is crowded with partial solutions and empty at the center, explained a decade of failures as sequencing and domain-scoping rather than refutation, and argued the window is closing as OS vendors absorb the semantics into closed features. The personal event bus is the missing middleware of user-owned computing — and the two companion papers in this program (P-002, P-003) are the consumer engines that make it necessary.

Appendix A — Survey method

Landscape claims (Table 1, the unoccupied-center claim) derive from structured web searches conducted 6–7 October 2026 via a commercial search API: ~80 queries across term sets {“personal event bus”, “ambient ledger”, “local-first personal data event stream”, “notification history app schema/export”, “personal data vault events”, “Solid pods events adoption”, “IFTTT local privacy alternative”, “n8n personal notifications”, “MyLifeBits”, “Databox personal data”}, English-language, first 10 results per query; inclusion criterion = the system claims to capture, normalize, or serve personal ambient events; exclusion = enterprise ETL/CEP/telemetry. Raw result JSONs are retained in the program’s research archive. Single-engine, English-only bias is acknowledged; the claim is “no shipping product found under this protocol,” not “no conceivable product exists.”

Disclosure

The p-rick program is developed alongside PocketVeto (this repository), cited here as an existence proof and case study. The bus is designed to be extractable from PocketVeto’s capture stack; the program’s interest in the category’s existence is therefore not disinterested, and readers should weigh the positioning accordingly.

References

- Culnan, M. J., & Armstrong, P. K. (1999). Information privacy concerns, procedural fairness, and impersonal trust. *Organization Science*, 10(1). — privacy calculus.
- Dienlin, T., & Trepte, S. (2015). Is the privacy paradox a relic of the past? *New Media & Society*.
- Haddadi, H., et al. (2016). Personal data management with the Databox. *ACM Cloud and Autonomic Computing Conference*. — closest academic precedent; dormant since ~2018.
- Kleppmann, M., Wiggins, A., van Hardenberg, S., & McGranaghan, M. (2019). *Local-First Software: You Own Your Data, in spite of the Cloud*. Ink & Switch. — local-first for documents; the event layer is absent.
- Searls, D., & ProjectVRM (2025). ProjectVRM — the customer-empowerment future is still not here (site posts, 2024–2025). projectvrn.org.
- Unver, B. (2017). The right to data portability in the GDPR and EU competition law. *European Data Protection Law Review* — Art. 20 scope analysis.
- Digital Markets Act (EU 2022/1925), Art. 6 — gatekeeper portability obligations.
- Android Developers. `NotificationListenerService` API reference. developer.android.com.
- Google Play policy (2019). SMS/Call Log permission restrictions. android-developers.googleblog.com.
- Weiser, M., & Brown, J. S. (1996). The Coming Age of Calm Technology. Xerox PARC. — ambient computing’s founding framing.
- Bell, G., & Gemmell, J. (2007). A Digital Life. *Scientific American*, 296(3); and the MyLifeBits project, Microsoft Research, 2001–2007. — durable personal capture without semantics or permissions.
- Dey, A. K., & Abowd, G. D. (2001). A conceptual framework and a toolkit for supporting the rapid prototyping of context-aware applications. *Human-Computer Interaction*, 16(2–4). — typed context subscriptions, research scale.
- Home Assistant (2026). Architecture: event bus, recorder, integration subscriptions. home-assistant.io/docs/architecture.
- Searls, D. (2025). ProjectVRM — post: the customer-empowerment future is still not here. projectvrn.org.
- Notification-history apps: Notlog (Play Store), Catch Notify (catch-notify.com). — raw-text capture, no semantics.
- PocketVeto (2025–2026). On-device payment-notification parsing, production evidence. github.com/srivtx/pocketveto.